

House Price Prediction

Using Machine Learning Regression Models

An End-to-End Regression Pipeline on the Ames Housing Dataset — from Data Cleaning and Feature Engineering to Explainable AI and Streamlit Deployment

PREPARED BY	Qiyas
DATASET	Ames Housing Dataset (1,460 Samples · 80 Features)
BEST MODEL	Tuned CatBoost Regressor — $R^2 = 0.9371$
DEPLOYMENT	Interactive Streamlit Application (Docker Containerized)

Table of Contents

1	Introduction.....	5
2	Problem Statement.....	5
3	Dataset Description.....	6
4	Data Cleaning.....	6
5	Exploratory Data Analysis (EDA).....	9
6	Feature Engineering.....	14
7	Regression Algorithms and Model Configuration.....	17
8	Hyperparameter Tuning.....	19
9	Model Evaluation.....	20
10	Model Comparison.....	23
11	Best Model Selection.....	24
12	Model Explainability.....	24
13	Deployment.....	29
14	Conclusion.....	30
15	Future Improvements.....	31

1 Introduction

Accurate house price prediction is an important task in the real estate industry because property prices depend on multiple interacting factors such as **location, property size, quality, age, and available facilities**.

Traditional property valuation methods often depend on manual inspection and expert judgment, which can be time-consuming and subjective. Machine learning provides a data-driven alternative by learning complex relationships between property characteristics and historical prices.

This project develops an **end-to-end machine learning regression system** for predicting residential property prices using the Ames Housing Dataset. The workflow covers data cleaning, exploratory data analysis, feature engineering, regression modeling, hyperparameter tuning, model evaluation, explainability analysis, and deployment through an interactive Streamlit application.

2 Problem Statement

Predicting residential house prices is a challenging regression problem because property values are influenced by many numerical and categorical factors. The objective of this project is to build a machine learning model that predicts house prices based on property characteristics.

The project aims to:

- Clean and preprocess housing data.
- Analyze patterns using exploratory data analysis.
- Engineer meaningful features.
- Train multiple regression algorithms.
- Optimize model performance through hyperparameter tuning.
- Evaluate and compare different models.
- Explain model predictions using feature importance and SHAP.
- Deploy the final system as an interactive web application.

Target Variable

- ✦ **SalePrice** — the final sale price of each residential property, in US dollars.

3 Dataset Description

This project uses the **Ames Housing Dataset**, a widely used benchmark dataset for regression tasks. The dataset contains detailed information about residential properties, including structural characteristics, quality ratings, neighborhood information, garage details, basement features, and other attributes that influence house prices.

Dataset Summary

Item	Value
Training Samples	1,460
Original Features	80
Target Variable	SalePrice

An initial inspection of the dataset using `train.info()` showed that the dataset consists of **38 numerical features** and **43 categorical features** (including the target variable). These features provide comprehensive information about each property and serve as predictors for estimating the selling price.

The dataset was also examined for duplicate records, and **no duplicate rows were found**, indicating that each observation represents a unique residential property.

Missing value analysis revealed that several features contained missing values. These missing values were primarily associated with optional property characteristics such as pools, fences, alleys, fireplaces, garages, and basements. Appropriate preprocessing techniques were later applied during the data cleaning stage to handle these missing values, ensuring that the dataset was suitable for model training and evaluation.

4 Data Cleaning

Data cleaning and preprocessing were performed to improve data quality and prepare the Ames Housing Dataset for machine learning. The dataset contained missing values, categorical variables, low-information features, and a small number of extreme observations that could negatively affect model performance. The following preprocessing steps were applied before model training.

4.1 Missing Value Handling

A detailed missing value analysis was conducted to identify incomplete features and determine appropriate imputation strategies based on the characteristics of each variable.

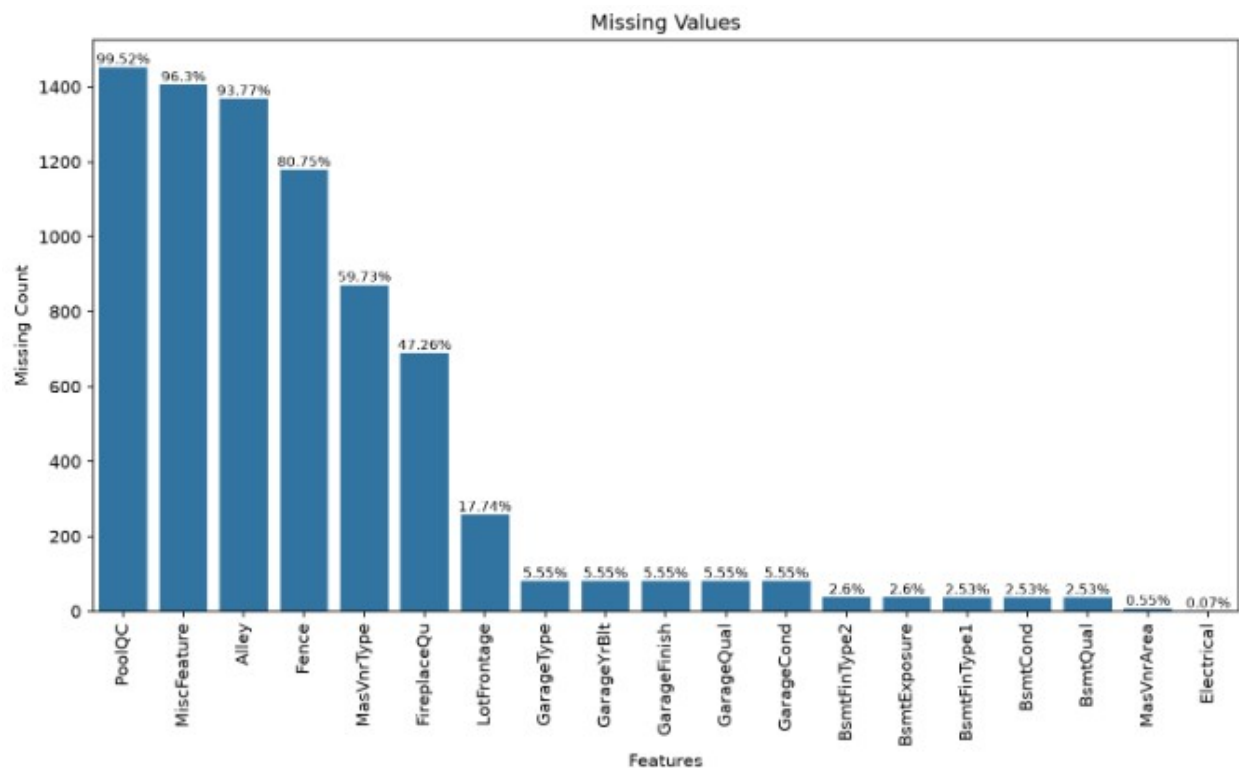


Figure 1. Missing values before cleaning.

Several features (**Alley**, **PoolQC**, **Fence**, and **MiscFeature**) contained more than **80% missing values**. Because these variables provided limited predictive value and had insufficient information for reliable imputation, they were removed from the dataset.

For the remaining missing values, different strategies were applied according to domain knowledge:

- Missing values in categorical features where the absence of a value indicated that the corresponding property feature did not exist were replaced with **“None”**. These features included:
 - MasVnrType
 - BsmtQual
 - BsmtCond
 - BsmtExposure
 - BsmtFinType1
 - BsmtFinType2
 - FireplaceQu
 - GarageType
 - GarageFinish
 - GarageQual

- GarageCond
- Numerical features containing missing values (**LotFrontage**, **MasVnrArea**, and **GarageYrBlt**) were imputed using the **median** value. Median imputation was selected because it is robust to extreme values and preserves the overall distribution of the data.
- The **Electrical** feature contained only a small number of missing values and was imputed using the **most frequent category (mode)**.
- Any remaining numerical features were also imputed using the median, while any remaining categorical features were filled with **“None”**, ensuring that the final dataset contained no missing values.

4.2 Duplicate Record Analysis

The dataset was examined for duplicate observations to ensure data quality and avoid bias during model training. The analysis confirmed that **no duplicate records were present**, so no duplicate removal was required.

4.3 Low-Information Feature Removal

Some categorical variables exhibited extremely low variation, with almost all observations belonging to a single category. Such features contribute little to prediction performance and may introduce unnecessary complexity. The following low-variance features were removed:

- Street
- Utilities
- Condition2
- RoofMatl

Removing these features simplified the dataset while retaining the variables with greater predictive potential.

4.4 Outlier Detection and Removal

A small number of extreme observations did not follow the general relationship between house size and selling price. Specifically, properties satisfying the following conditions were treated as outliers:

- **Ground Living Area (GrLivArea) greater than 4,000 square feet**
- **SalePrice less than \$300,000**

These observations represented unusually large houses sold at comparatively low prices and could disproportionately influence regression models. Consequently, they were removed prior to model training to improve model stability and predictive performance.

4.5 Feature Encoding

Most machine learning algorithms require numerical input; therefore, categorical variables were converted into numerical representations during preprocessing.

The project employed **One-Hot Encoding**, which creates a separate binary indicator for each category while avoiding the introduction of artificial ordinal relationships between categorical values.

To improve robustness during prediction, the encoder was configured with `handle_unknown="ignore"`, allowing the model to safely process categories that were not observed during training.

4.6 Feature Transformation

After cleaning the dataset, an automated preprocessing pipeline was constructed using Scikit-learn's **ColumnTransformer** and **Pipeline** utilities. The preprocessing pipeline applied different transformations based on feature type:

Numerical Features

- Median imputation for missing values.
- Standardization using **StandardScaler** for algorithms sensitive to feature magnitude, including Linear Regression, Ridge Regression, and Lasso Regression.

Categorical Features

- Missing values imputed using the most frequent category where required.
- One-Hot Encoding to transform categorical variables into machine-learning-compatible numerical features.

Tree-based algorithms such as Decision Tree, Random Forest, Gradient Boosting, and CatBoost were trained without feature scaling because they are inherently insensitive to differences in feature scale.

4.7 Data Preparation

Following data cleaning and preprocessing, the dataset contained no duplicate records, all missing values had been appropriately handled, low-information features had been removed, and categorical variables were successfully encoded.

The cleaned dataset was then divided into training and testing sets, providing a reliable basis for model training, evaluation, and comparison.

5 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed on the cleaned dataset to understand the distribution of house prices, identify important patterns, analyze relationships between features, and guide feature engineering and model development decisions.

The analysis focused on understanding the target variable (SalePrice), identifying influential features, and investigating relationships between numerical and categorical variables.

5.1 SalePrice Distribution Analysis

The distribution of the target variable was analyzed to understand the range, spread, and characteristics of residential property prices.

The analysis revealed the overall price distribution and helped identify whether the target variable contained skewness. Since house prices commonly exhibit right-skewed distributions, a log transformation was also analyzed to improve visualization and understand the effect of transformation on the target distribution.

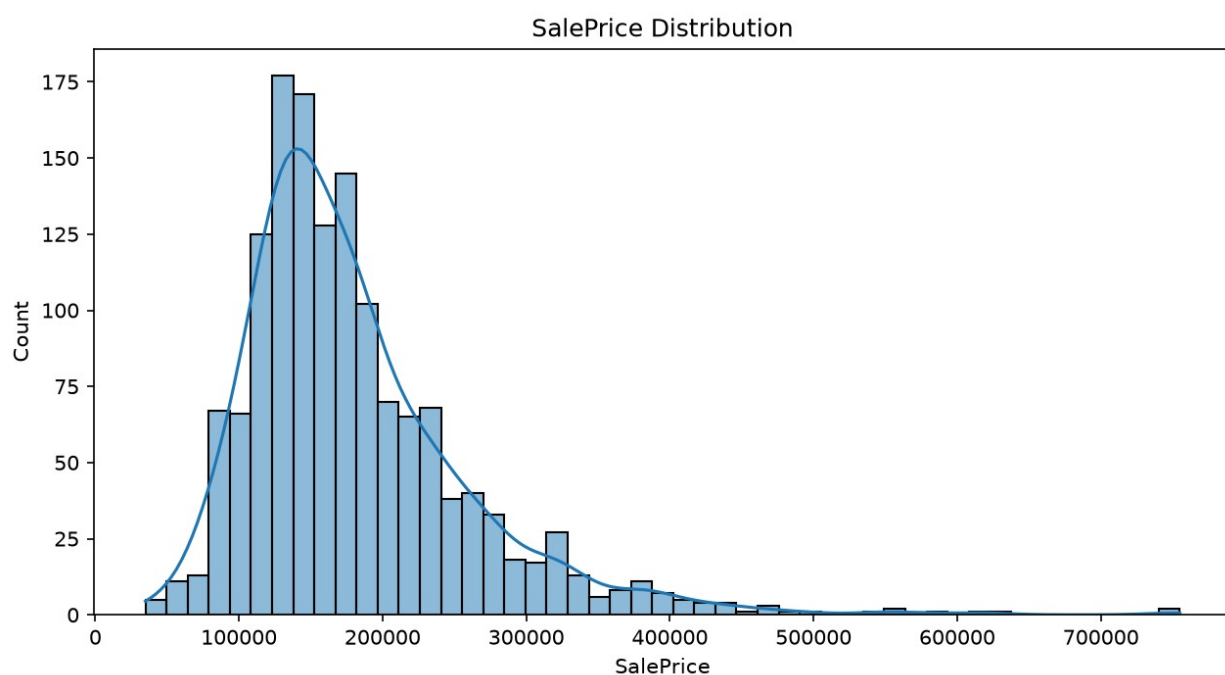


Figure 2. SalePrice distribution.

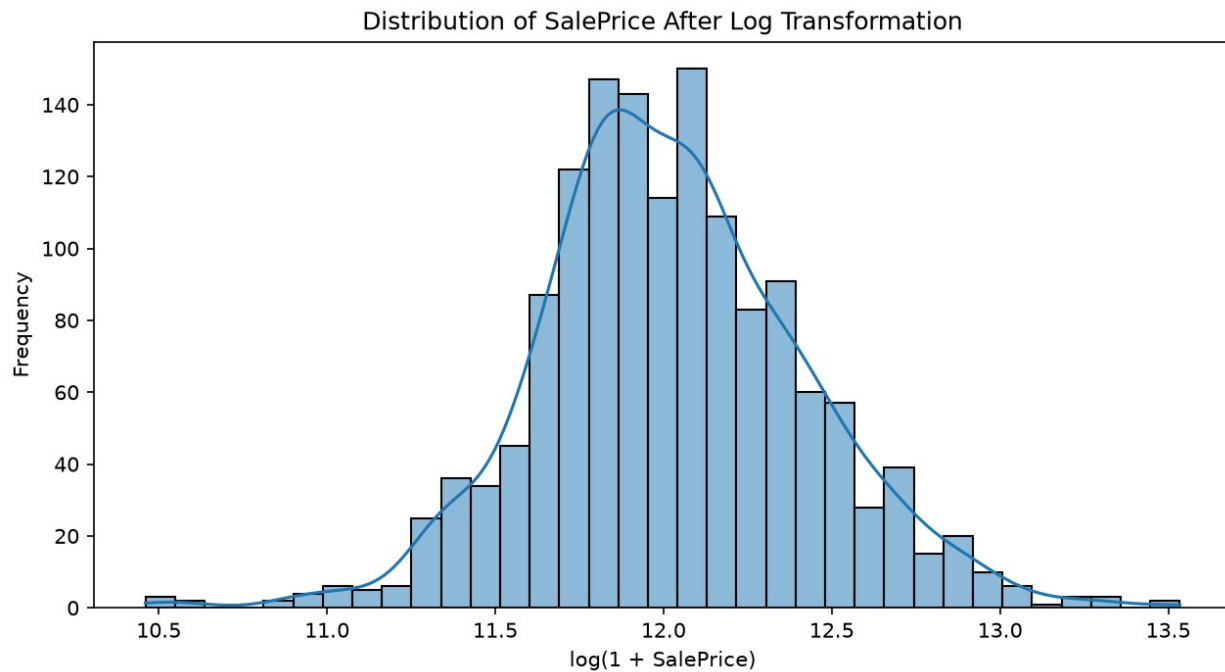


Figure 3. Log-transformed SalePrice distribution.

5.2 Correlation Analysis

Correlation analysis was conducted to identify numerical features with strong relationships with the target variable. A correlation heatmap was generated to visualize relationships between numerical variables and identify potential predictors of house prices.

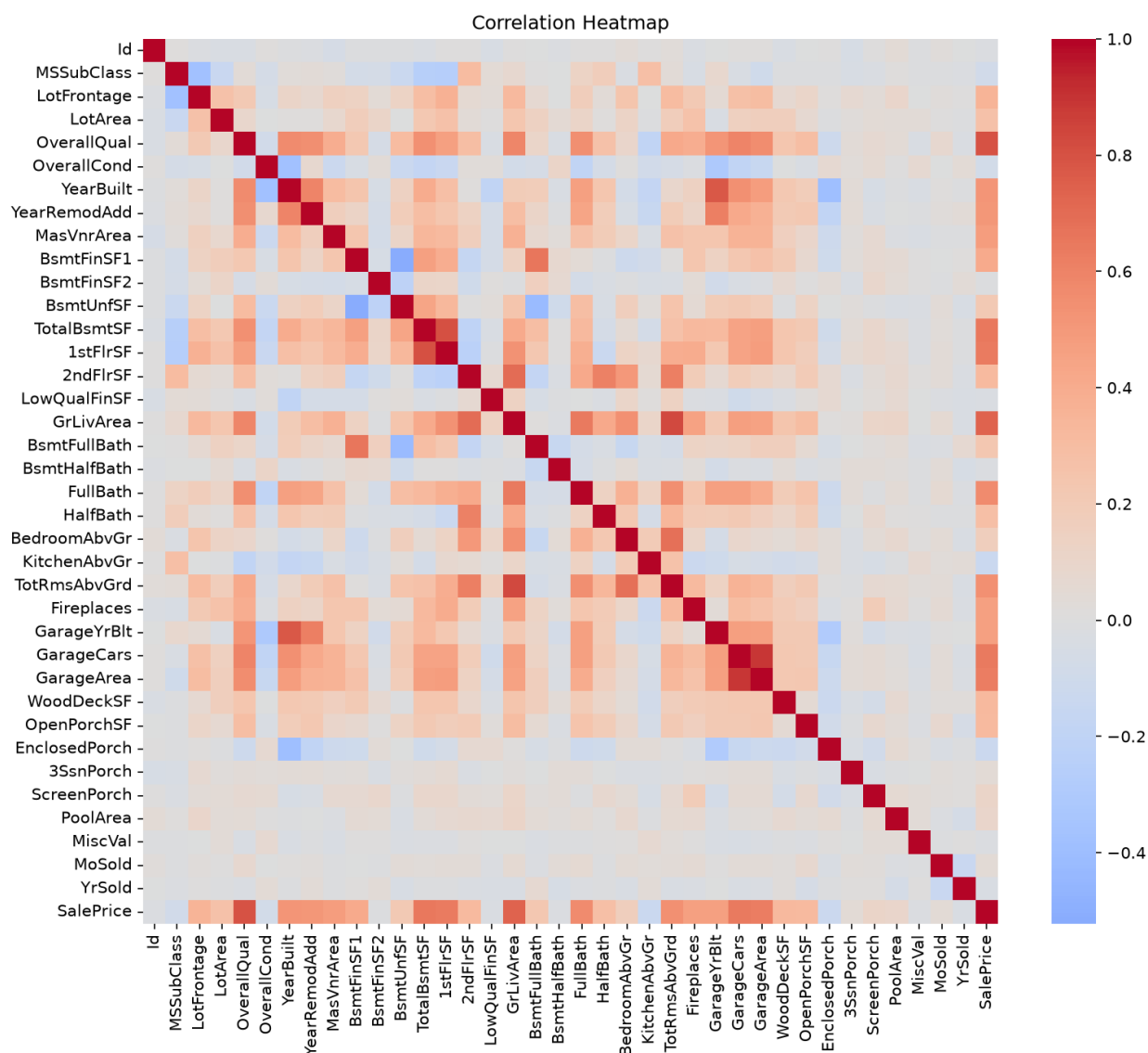


Figure 4. Correlation heatmap of numerical features.

The analysis showed that several features had strong relationships with SalePrice, including:

- **OverallQual** — Overall material and finish quality of the house.
- **GrLivArea** — Above-ground living area in square feet.
- **TotalBsmtSF** — Total basement area.
- **GarageCars** — Garage capacity measured by number of cars.

These features were identified as important predictors and were considered during feature engineering and model training.

5.3 Numerical Feature Relationship Analysis

The relationship between important numerical features and house prices was analyzed using visualization techniques such as scatter plots. These plots helped identify linear and non-linear relationships, trends between property characteristics and selling price, and potential remaining extreme observations.

Examples of analyzed numerical features include:

- OverallQual vs SalePrice
- GrLivArea vs SalePrice
- TotalBsmtSF vs SalePrice
- GarageCars vs SalePrice

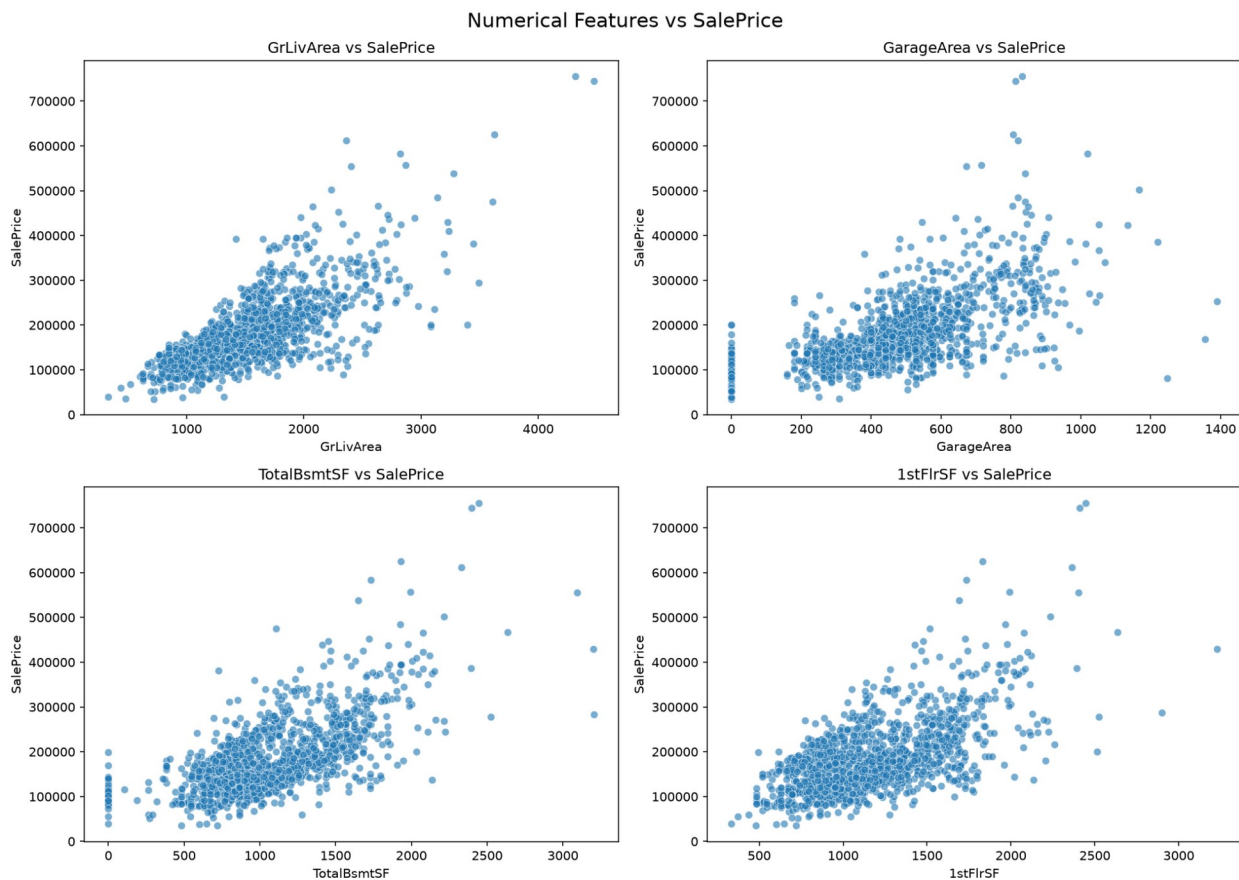


Figure 5. Numerical features vs SalePrice.

5.4 Categorical Feature Relationship Analysis

Categorical features were analyzed to understand how different property categories influence house prices. Box plots and categorical visualizations were used to compare price distributions across different groups. Important categorical variables analyzed include:

- Neighborhood

- KitchenQual
- Exterior Quality
- GarageFinish
- BsmtQual

These analyses showed that property quality indicators and location-related features significantly affect house prices.

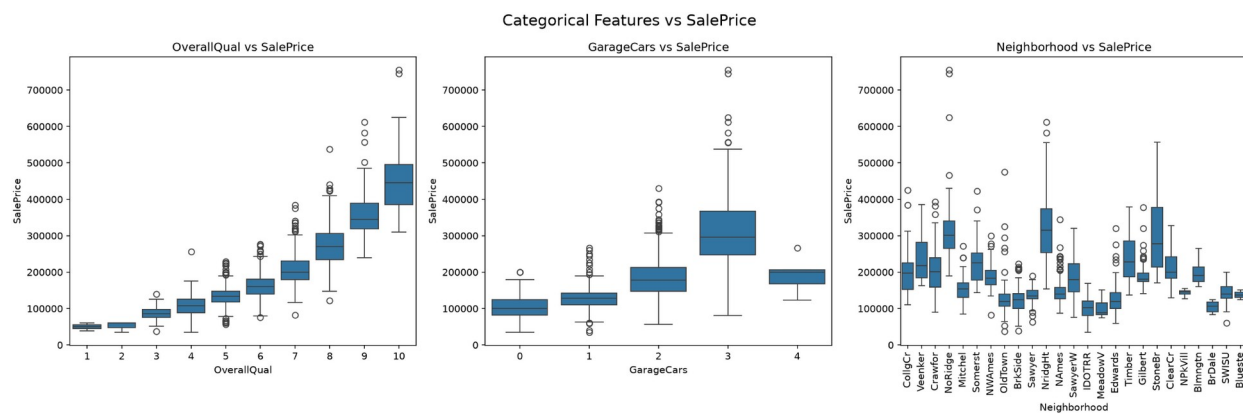


Figure 6. Categorical features vs SalePrice.

5.5 Key EDA Findings

What the Data Told Us

- ◆ House quality (OverallQual) was one of the strongest predictors of property value.
- ◆ Larger living areas generally corresponded to higher house prices.
- ◆ Properties with better-quality materials, garages, and finished basements tended to have higher selling prices.
- ◆ Neighborhood and quality-related attributes showed significant differences in price distributions.

The findings from EDA guided the feature engineering process and helped determine which features should be prioritized during model development.

6 Feature Engineering

Feature engineering was performed to extract additional information from the original housing attributes and create more meaningful predictors for the regression models. The objective was to represent important aspects of a property, such as size, age, quality, and additional facilities, in a more informative way.

6.1 Total Square Footage Feature

A new feature, **TotalSF**, was created to represent the overall size of the property. It combines:

- TotalBsmtSF (Basement area)
- 1stFlrSF (First-floor area)
- 2ndFlrSF (Second-floor area)

$$TotalSF = TotalBsmtSF + 1stFlrSF + 2ndFlrSF$$

This feature provides a more complete representation of the total usable area of a house compared with individual floor measurements.

6.2 Total Bathroom Feature

A weighted bathroom feature was created by combining full and half bathrooms from both above-ground and basement areas. The feature was calculated as:

$$TotalBathrooms = FullBath + 0.5 \times HalfBath + BsmtFullBath + 0.5 \times BsmtHalfBath$$

This transformation allows the model to capture the overall bathroom availability of a property using a single continuous feature.

6.3 House Age Features

Property age was engineered to capture the effect of construction history on house prices. Two new features were created:

HouseAge

$$HouseAge = YrSold - YearBuilt$$

This represents the age of the house at the time of sale.

YearsSinceRemodel

$$YearsSinceRemodel = YrSold - YearRemodAdd$$

These features help the model understand how newer construction and recent renovations influence property value.

6.4 Outdoor Living Area Feature

A combined outdoor space feature, **TotalPorchSF**, was created by aggregating different outdoor areas:

- OpenPorchSF
- EnclosedPorch
- 3SsnPorch
- ScreenPorch
- WoodDeckSF

This provides a single measurement of additional outdoor living space, which may contribute to property value.

6.5 Quality and Living Area Interaction Feature

A feature interaction was created between overall property quality and living area, named **QualityLivingArea**:

$$QualityLivingArea = OverallQual \times GrLivArea$$

This captures the combined effect of house size and construction quality — a large house with poor quality may not have the same value as a similarly sized high-quality property.

6.6 Garage Value Feature

A garage-related feature, **GarageScore**, was created by combining garage size and capacity:

$$GarageScore = GarageCars \times GarageArea$$

This represents both the functional capacity and physical size of the garage.

6.7 Feature Engineering Impact

The engineered features provide additional information that is not directly available from the original variables. Instead of relying only on individual measurements, the model receives higher-level property characteristics representing total property size, bathroom capacity, property age, renovation history, outdoor facilities, quality-size interaction, and garage contribution.

Original Features	Engineered Feature
TotalBsmtSF + 1stFlrSF + 2ndFlrSF	TotalSF
FullBath + HalfBath + Bsmt Bath Features	TotalBathrooms
YrSold - YearBuilt	HouseAge
YrSold - YearRemodAdd	YearsSinceRemodel
Porch and Deck Features	TotalPorchSF
OverallQual × GrLivArea	QualityLivingArea

Original Features	Engineered Feature
GarageCars × GarageArea	GarageScore

7 Regression Algorithms and Model Configuration

A total of **twelve regression algorithms** were implemented and evaluated. The models included linear regression approaches, tree-based models, ensemble methods, and gradient boosting algorithms. Each model was trained using the same preprocessing pipeline to ensure a fair comparison.

Linear Regression

Used as the baseline model.

Configuration

- Algorithm: LinearRegression
- Regularization: None
- Parameters: default scikit-learn configuration

Ridge Regression

Applies L2 regularization to reduce overfitting and improve stability with correlated features.

Configuration

- Algorithm: Ridge Regression
- alpha = 1.0

Lasso Regression

Applies L1 regularization to reduce unnecessary feature contributions.

Configuration

- Algorithm: Lasso Regression
- alpha = 0.001
- max_iter = 20000

Decision Tree Regressor

Learns nonlinear relationships through recursive feature-based splitting.

Configuration

- Algorithm: DecisionTreeRegressor
- random_state = 42

Random Forest Regressor

Combines multiple decision trees to reduce variance and improve generalization.

Configuration

- Algorithm: RandomForestRegressor
- n_estimators = 200
- random_state = 42
- n_jobs = -1

Extra Trees Regressor

Introduces additional randomness during tree construction to improve generalization.

Configuration

- Algorithm: ExtraTreesRegressor
- n_estimators = 200
- random_state = 42
- n_jobs = -1

Gradient Boosting Regressor

Builds sequential models where each learner improves previous prediction errors.

Configuration

- Algorithm: GradientBoostingRegressor
- random_state = 42

AdaBoost Regressor

Improves prediction by focusing more on difficult training samples.

Configuration

- Algorithm: AdaBoostRegressor
- random_state = 42

Support Vector Regressor (SVR)

Uses kernel methods to model nonlinear relationships between housing features and prices.

Configuration

- Algorithm: SVR
- kernel = "rbf"

XGBoost Regressor

An optimized gradient boosting algorithm designed for structured regression problems.

Configuration

- Algorithm: XGBRegressor
- n_estimators = 400

- learning_rate = 0.03
- max_depth = 3
- min_child_weight = 5
- subsample = 0.7
- colsample_bytree = 0.7
- random_state = 42

LightGBM Regressor

Provides efficient gradient boosting using tree-based learning.

Configuration

- Algorithm: LGBMRegressor
- n_estimators = 500
- learning_rate = 0.03
- random_state = 42

CatBoost Regressor

A gradient boosting algorithm optimized for categorical and tabular datasets.

Configuration

- Algorithm: CatBoostRegressor
- iterations = 500
- learning_rate = 0.03
- depth = 4
- l2_leaf_reg = 5
- random_state = 42
- verbose = 0

8 Hyperparameter Tuning

Hyperparameter optimization was performed on the CatBoost model using **GridSearchCV** with **5-fold cross-validation**. The following parameters were optimized:

Parameter	Search Values
Iterations	500
Learning Rate	0.03, 0.05, 0.07
Tree Depth	3, 4

Parameter	Search Values
L2 Regularization	3, 5

The best configuration was selected based on the highest cross-validation R^2 score. The optimized model achieved improved performance compared with the original CatBoost configuration.

9 Model Evaluation

The performance of all regression models was evaluated using four standard regression metrics:

- **Mean Absolute Error (MAE)** — measures the average absolute difference between the actual and predicted house prices. A lower MAE indicates that the model produces predictions closer to the true values.
- **Mean Squared Error (MSE)** — measures the average squared difference between actual and predicted prices. It gives greater importance to larger prediction errors.
- **Root Mean Squared Error (RMSE)** — represents the average prediction error in the original price scale. It is particularly useful for interpreting the practical accuracy of house price predictions.
- **R^2 Score** — measures the proportion of variance in house prices explained by the model. Higher R^2 values indicate better predictive performance and stronger ability to capture relationships within the dataset.

9.1 Actual vs Predicted Analysis

The Actual vs Predicted plot was used to visually evaluate the accuracy of the final regression model. The closer the predictions are to the diagonal reference line, the better the agreement between predicted and actual house prices.

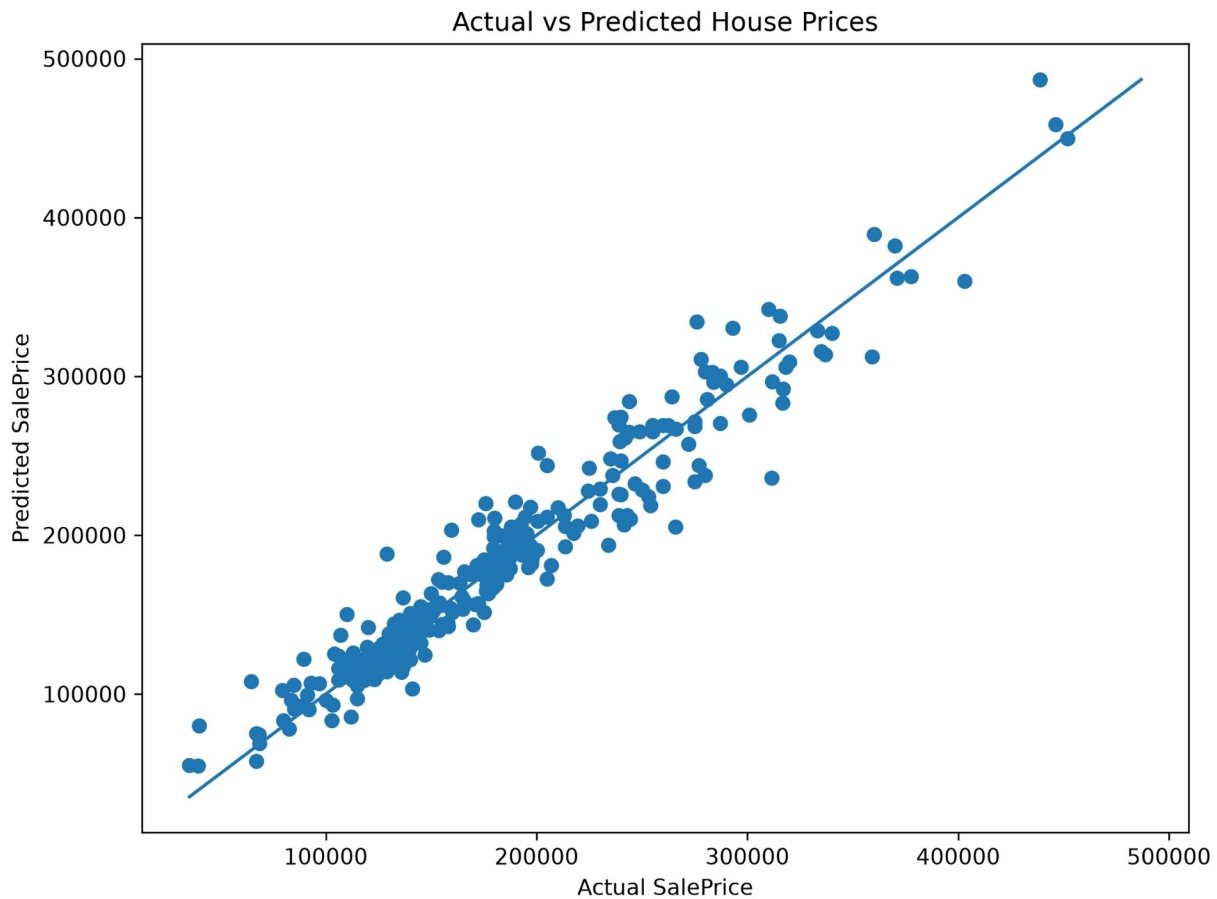


Figure 7. Actual vs predicted house prices.

9.2 Residual Analysis

Residual analysis was performed to investigate the distribution and behavior of prediction errors. Residuals were calculated as:

$$\text{Residual} = \text{Actual Value} - \text{Predicted Value}$$

A well-performing regression model should have residuals randomly distributed around zero. This indicates that the model has captured the underlying patterns in the housing data without systematic prediction errors.

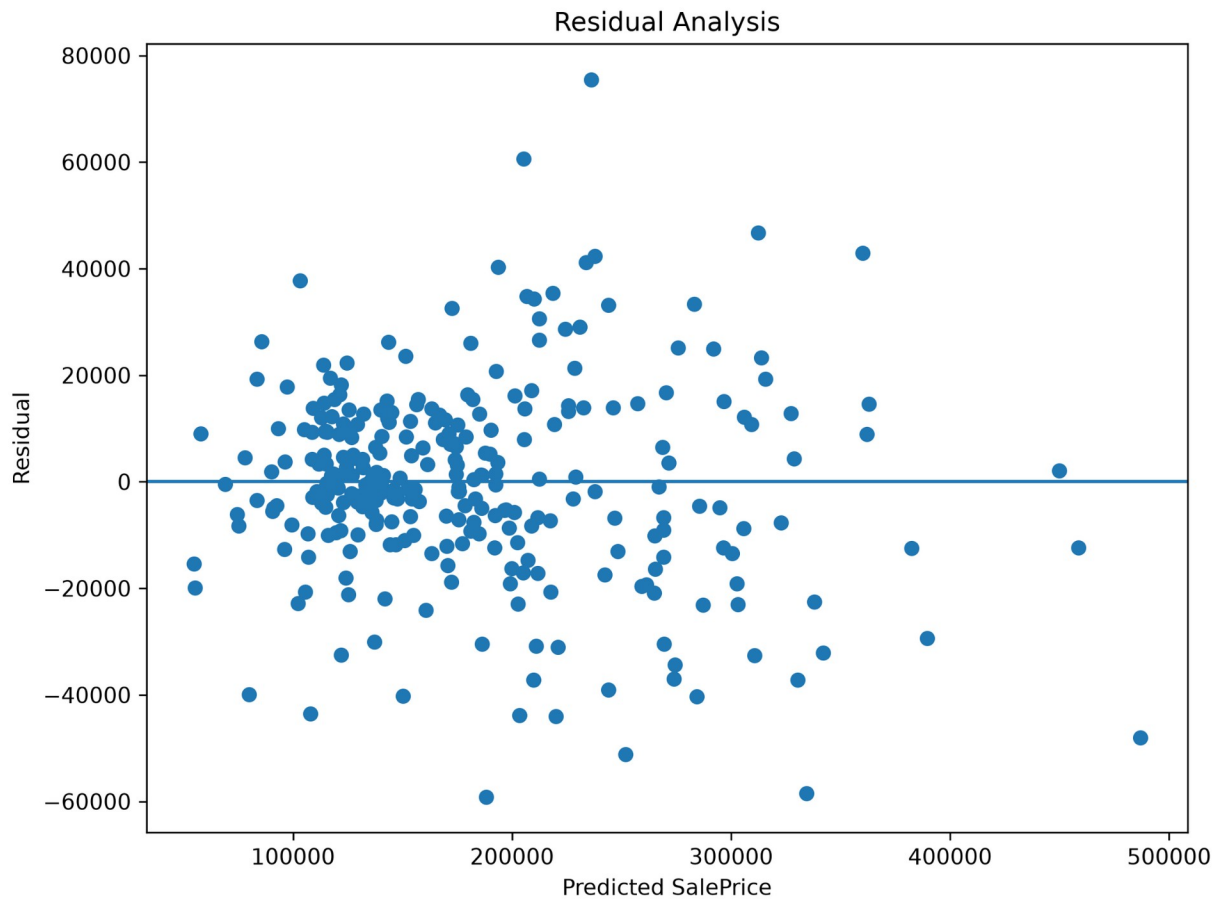


Figure 8. *Residual analysis plot.*

9.3 Learning Curve Analysis

A learning curve was used to evaluate the model's generalization ability and identify potential overfitting or underfitting issues. The curve compares training and validation performance as the training dataset size increases. A smaller difference between training and validation scores indicates better generalization, while a large gap may indicate overfitting.

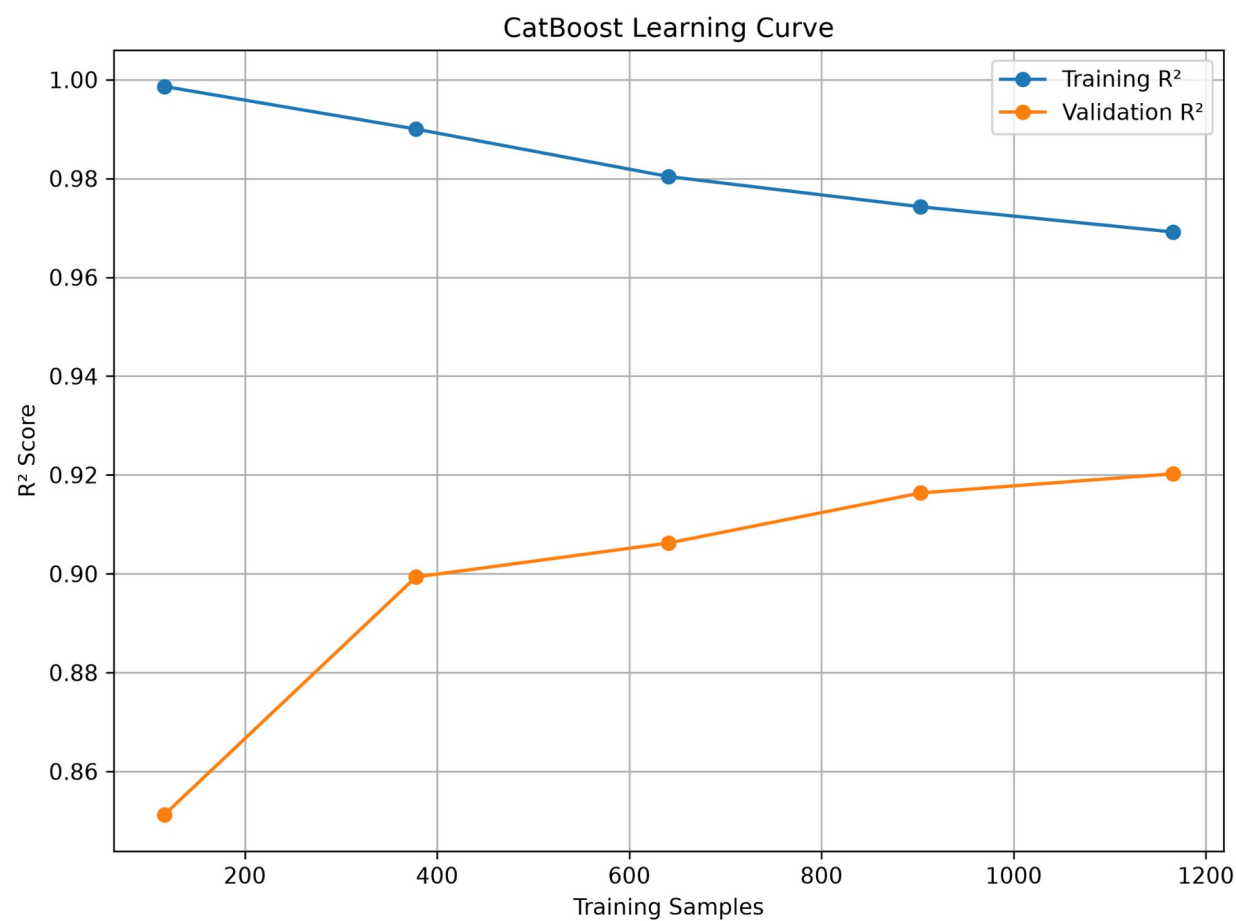


Figure 9. Learning curve analysis.

10 Model Comparison

All regression models were evaluated using MAE, MSE, RMSE, R^2 Score, and Training Time. Lower MAE, MSE, and RMSE values indicate better performance, while a higher R^2 score indicates better explanatory power.

Model	MAE	MSE	RMSE	R^2 Score	Training Time
Tuned CatBoost ★	13,680.99	347,307,581.29	18,636.19	0.9371	4.2627s
CatBoost	13,721.36	352,137,210.70	18,765.32	0.9363	5.2407s
Gradient Boosting	14,188.29	362,344,564.19	19,035.35	0.9344	2.6105s
XGBoost	14,069.38	369,351,646.11	19,218.52	0.9331	1.2946s
Lasso	14,013.64	371,676,672.33	19,278.92	0.9327	0.5063s

Model	MAE	MSE	RMSE	R ² Score	Training Time
Regression					
Ridge Regression	14,377.74	408,626,733.67	20,214.52	0.9260	0.1909s
LightGBM	14,712.11	432,801,062.15	20,803.87	0.9216	6.0868s
Linear Regression	14,865.40	450,975,579.24	21,236.19	0.9184	0.3757s
Extra Trees	15,317.45	457,128,600.61	21,380.57	0.9172	7.0665s
SVR	15,519.32	492,584,991.77	22,194.26	0.9108	0.4555s
Random Forest	16,120.74	536,420,239.57	23,160.75	0.9029	6.5196s
AdaBoost	20,757.29	765,441,849.09	27,666.62	0.8614	2.1640s
Decision Tree	23,074.37	1,115,518,626.01	33,399.38	0.7980	0.5020s

11 Best Model Selection

Based on the evaluation results, the **Tuned CatBoost Regressor** was selected as the final model.

Final Model Performance — Tuned CatBoost Regressor

- ✦ Lowest RMSE: **18,636.19**
- ✦ Lowest MAE: **13,680.99**
- ✦ Highest R² Score: **0.9371**

Compared with the original CatBoost model, hyperparameter tuning improved performance by reducing prediction error and slightly increasing explained variance.

The final tuned model was saved and integrated into the Streamlit prediction application.

12 Model Explainability

Although machine learning models can achieve high predictive performance, understanding the factors influencing their predictions is important for transparency and trust. Model explainability techniques were applied to analyze how the selected model makes house price predictions and to identify the most influential features.

12.1 Feature Importance Analysis

Feature importance analysis was performed to identify the housing characteristics that contribute most to the model's predictions. The analysis was based on the feature importance values extracted from the tree-based final model. Higher importance values indicate that a feature has a greater influence on the model's decision-making process. The most influential features included:

- **OverallQual** — Overall material and finish quality of the house.
- **GrLivArea** — Above-ground living area in square feet.
- **TotalSF** — Combined total house area including basement and floor areas.
- **QualityLivingArea** — Interaction between overall quality and living area.
- **GarageScore** — Combined garage capacity and area information.
- **HouseAge** — Age of the property at the time of sale.

These results indicate that both structural characteristics and quality-related features play a major role in determining house prices.

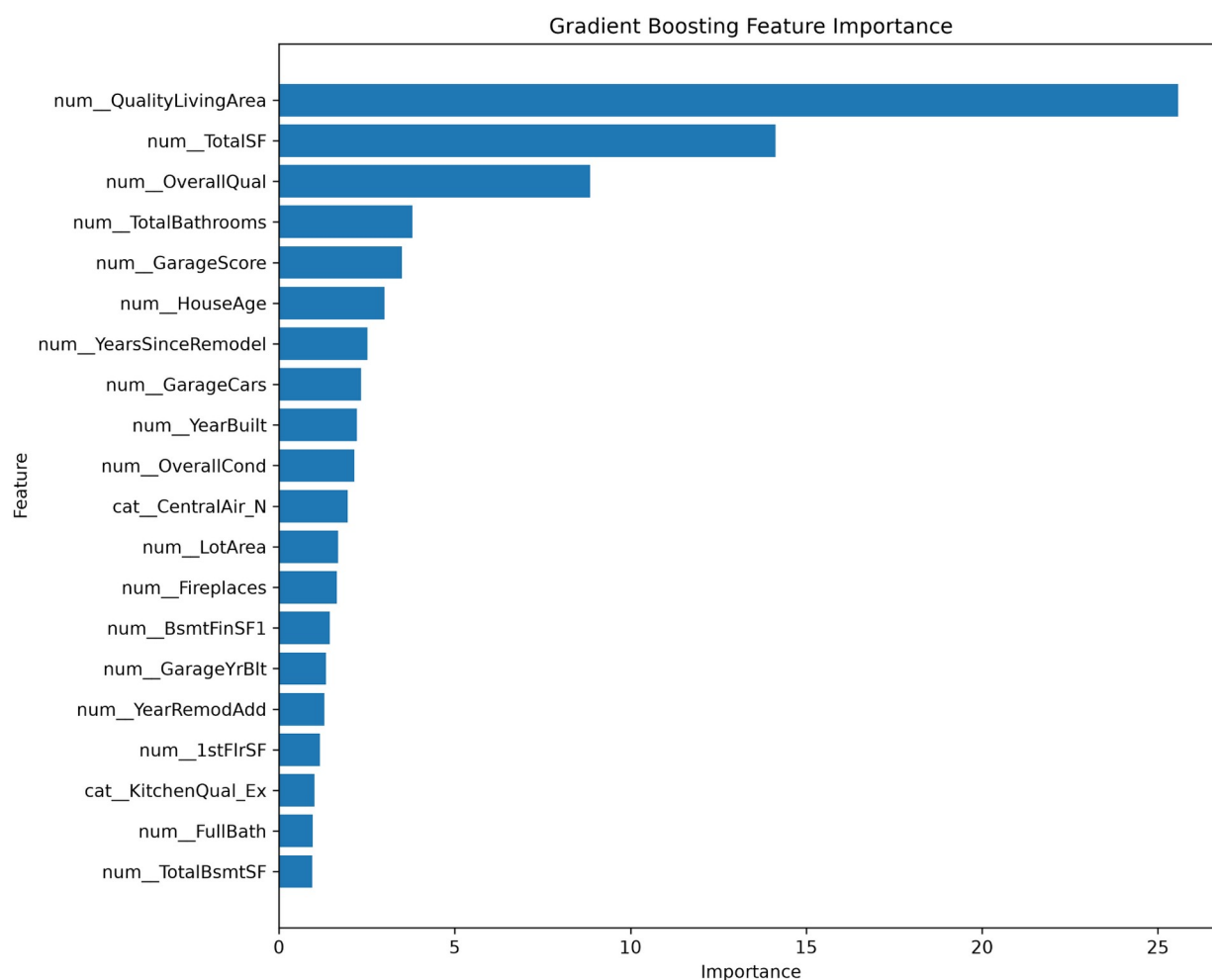


Figure 10. Feature importance analysis.

12.2 SHAP Explainability Analysis

SHAP (SHapley Additive exPlanations) was applied to provide a deeper understanding of model predictions. SHAP explains how individual features contribute to increasing or decreasing the predicted house price. The SHAP analysis provides:

- **Global feature importance** — identifies the features with the greatest overall impact on model predictions.
- **Feature contribution direction** — shows whether a feature increases or decreases the predicted price.
- **Individual prediction explanations** — explains why a specific house received a particular prediction value.

SHAP Feature Importance Plot

The SHAP bar plot summarizes the average contribution of each feature across the dataset. Features with higher mean absolute SHAP values have a greater influence on the model output.

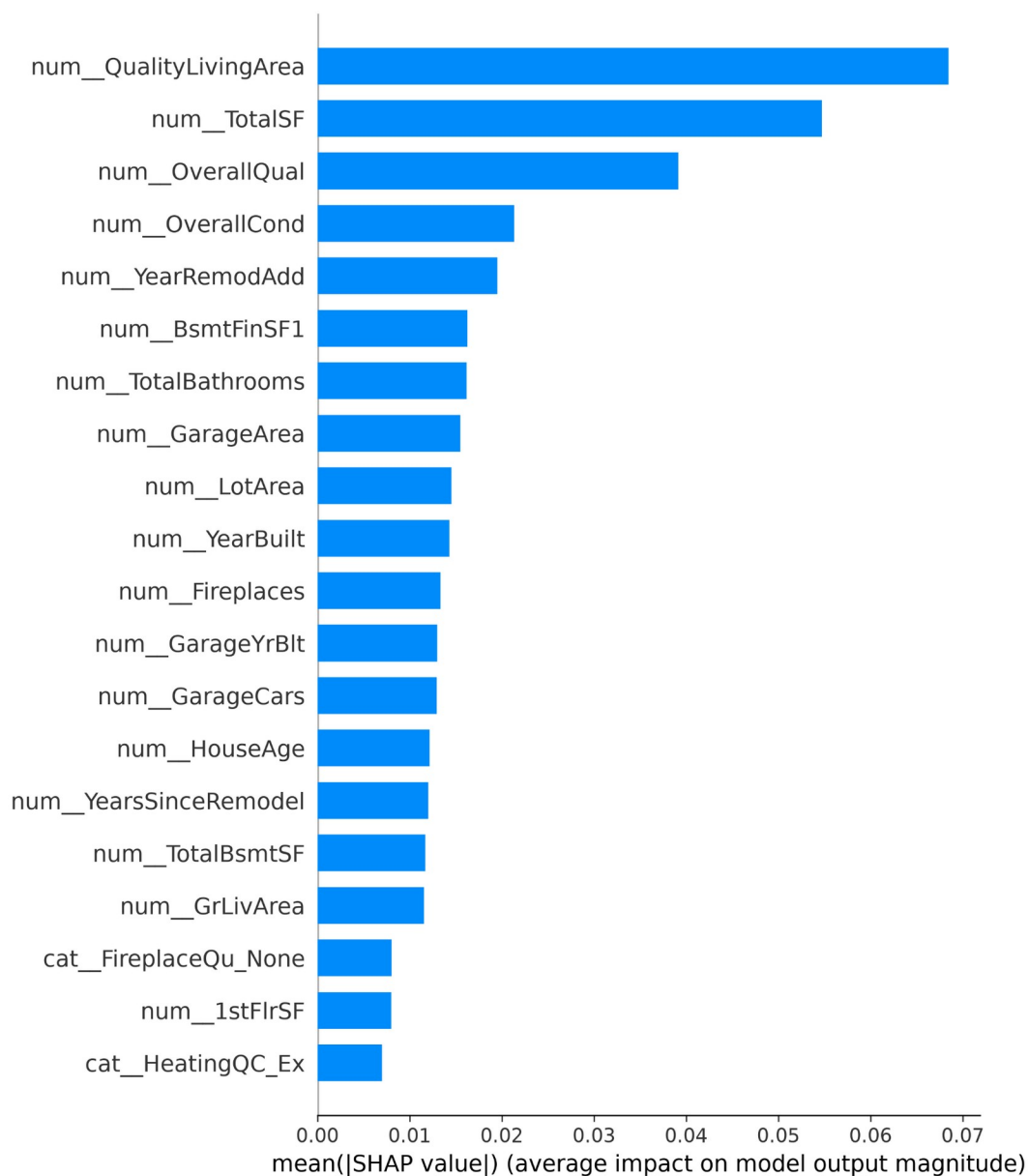


Figure 11. SHAP feature importance plot.

SHAP Summary Plot

The SHAP summary plot provides both feature importance and the direction of influence. It shows how high and low feature values affect the predicted house prices. For example, higher values of important features such as overall quality and living area generally contribute positively to predicted prices.

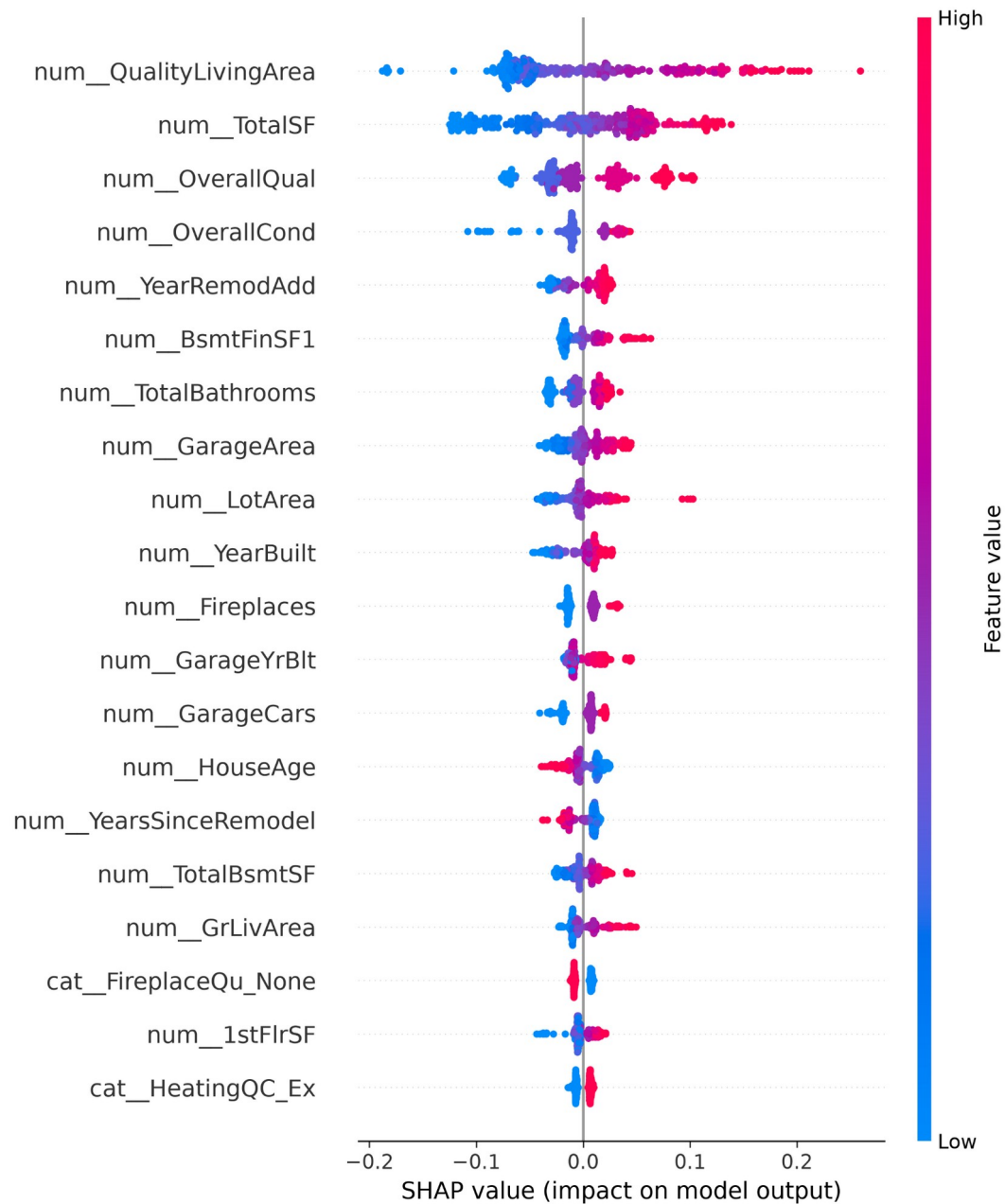


Figure 12. SHAP summary plot.

SHAP Waterfall Plot

The SHAP waterfall plot explains individual predictions by showing how each feature contributes to moving the prediction from the model's baseline value to the final predicted house price. Positive SHAP values indicate features that increase the predicted price, while negative SHAP values indicate features that decrease the prediction.

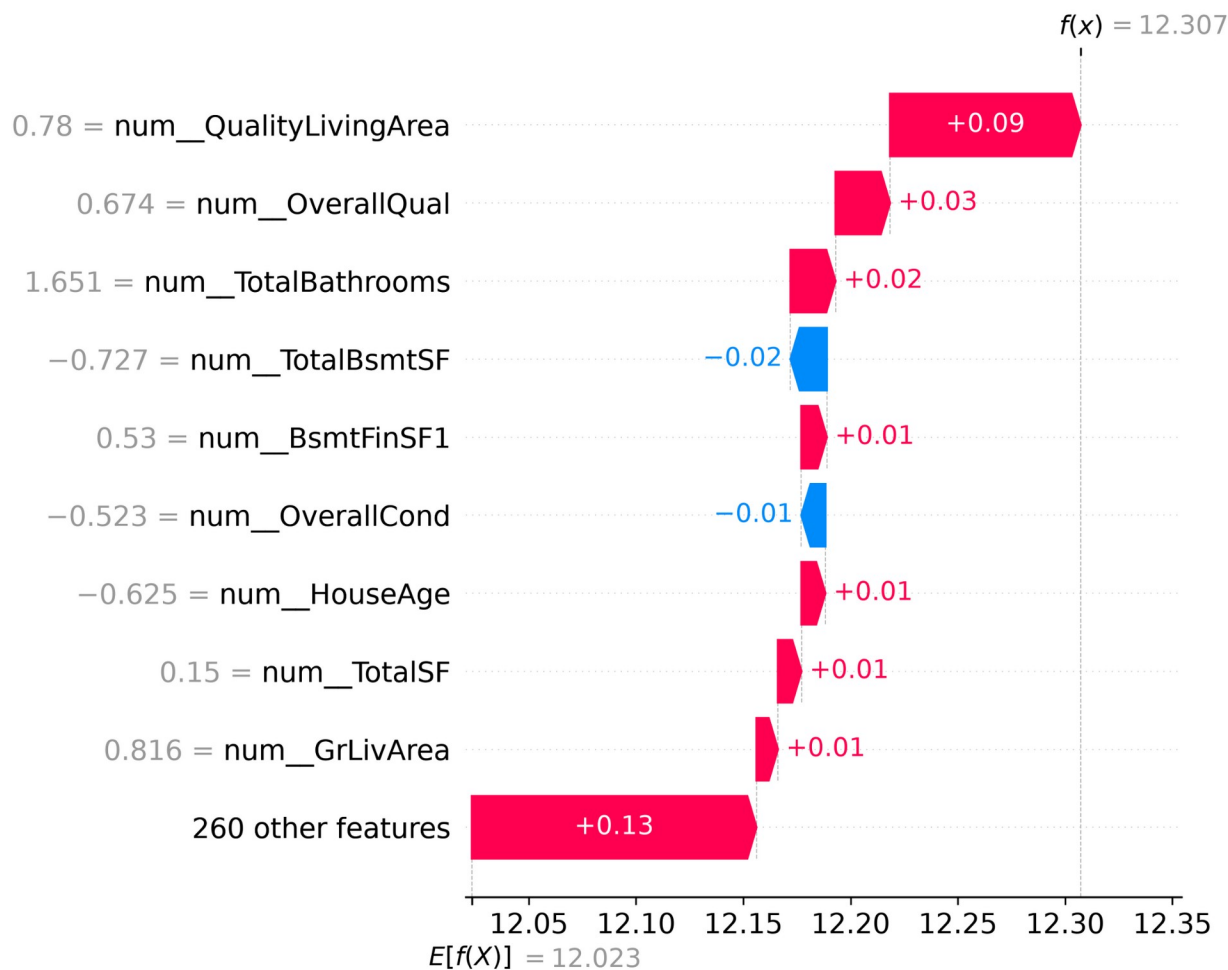


Figure 13. SHAP waterfall explanation.

13 Deployment

The final house price prediction model was deployed as an interactive web application using **Streamlit**. The application provides a user-friendly interface that allows users to explore the dataset, analyze model performance, understand prediction factors, and generate real-time house price predictions.

The deployed application includes the following functionalities:

- **Dataset Exploration** — allows users to view dataset information, statistics, and sample records.
- **Exploratory Data Analysis (EDA)** — provides interactive visualizations including target distribution, correlation analysis, and feature relationships.
- **Model Evaluation Dashboard** — displays model performance metrics, comparison results, and prediction analysis plots.
- **Feature Importance Analysis** — shows the most influential housing features affecting model predictions.

- **SHAP Explainability** — provides interpretable explanations of model behavior and individual predictions.
- **Interactive House Price Prediction** — allows users to enter house characteristics and receive predicted sale prices from the trained model.

The deployment workflow involved:

- **Streamlit Cloud** for hosting the interactive machine learning application.
- **Docker Containerization** for packaging the application environment and ensuring reproducible deployment.
- **Docker Hub** for storing and managing the Docker image.

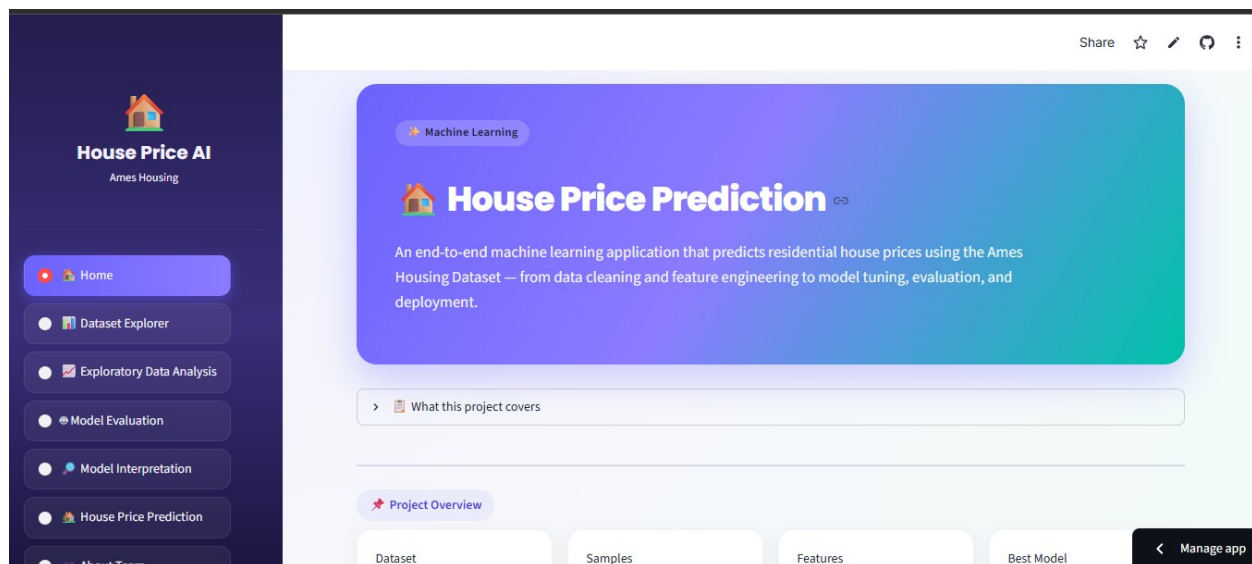


Figure 14. Streamlit application screenshot.

14 Conclusion

This project successfully developed an end-to-end machine learning regression system for residential house price prediction using the Ames Housing Dataset. The project covered the complete machine learning workflow, including:

- Data understanding and preparation
- Data cleaning and preprocessing
- Exploratory data analysis
- Feature engineering
- Regression model development
- Hyperparameter optimization
- Model evaluation and comparison

- Model explainability
- Application deployment

Multiple regression algorithms were evaluated, and the best-performing model was selected based on regression performance metrics. Explainability techniques such as feature importance analysis and SHAP were applied to improve transparency and understand the factors influencing predictions.

The final system provides both accurate house price predictions and interpretable insights into model decision-making, demonstrating a complete machine learning solution from data processing to deployment.

15 Future Improvements

Although the developed system achieved strong predictive performance, several improvements can be explored in future work:

- **Advanced Hyperparameter Optimization** — apply optimization techniques such as Bayesian Optimization, Optuna, or automated search strategies to further improve model performance.
- **Cloud Deployment Expansion** — deploy the application using cloud platforms such as AWS, Microsoft Azure, or Google Cloud for improved scalability and accessibility.
- **Automated Machine Learning (AutoML)** — integrate AutoML frameworks to automatically explore preprocessing strategies, feature engineering methods, and model selection.
- **Model Monitoring and Maintenance** — implement monitoring systems to track model performance, detect data drift, and schedule model updates.
- **Real-Time Housing Market Integration** — integrate external housing market data sources to improve prediction accuracy with updated market trends.
- **MLOps Implementation** — develop a complete MLOps pipeline including automated training, testing, version control, model registry, and continuous deployment.